

RELATÓRIO TÉCNICO – PRÉ-PROCESSAMENTO

PROJETO CARDIO-IA (FASE 4)

Turma: 2TIAOB

Alunos

Nicolas Antonio Silva Araujo - RM 566307

Vitória Pereira Bagatin - RM 566519

Link do repositório no Github: [lab-ia-shared/Cardio-IA-4](https://github.com/lab-ia-shared/Cardio-IA-4)

1. Introdução e Contextualização do Dataset

O projeto **Cardio-IA (Fase 4)** evoluiu do monitoramento IoT em tempo real para o suporte à decisão clínica utilizando **Visão Computacional**. Para manter o foco cardiovascular, selecionamos o dataset público *Cardiomegaly Disease Prediction* (derivado do *NIH Chest X-rays*) para identificar **Cardiomegalia** (aumento do coração) em radiografias de tórax. O conjunto possui **5.552 imagens** divididas de forma binária entre as classes **false** (normal) e **true** (cardiomegalia).

2. Pipeline de Pré-processamento

Para padronizar a entrada do modelo de Deep Learning, as imagens passaram por três etapas matemáticas fundamentais:

- **Conversão para RGB (3 canais):** As radiografias originais em escala de cinza (1 canal) foram convertidas para o padrão RGB. Isso garante compatibilidade nativa com as arquiteturas de *Transfer Learning* (VGG16/ResNet) exigidas na próxima etapa.
- **Redimensionamento (224 \times 224):** As imagens originais de 128 \times 128 pixels sofreram um *upscaling* geométrico para o padrão de 224 \times 224 pixels, tamanho padrão de entrada da maioria das redes convolucionais.
- **Normalização ([0.0, 1.0]):** Os pixels (0 a 255) foram escalonados para o intervalo [0.0, 1.0] dividindo-se por 255. Essa operação acelera a convergência do Gradiente Descendente e evita instabilidade numérica nos pesos da rede.

Justificativa Teórica: A normalização é indispensável porque impede que os gradientes saturem durante o algoritmo de *Gradient Descent* (Gradiente Descendente). Manter os valores próximos de zero suaviza a função de perda (*loss function*), acelera drasticamente a convergência do treinamento e evita instabilidade numérica nos pesos sinápticos.

3. Gerenciamento de Memória RAM e Arquitetura de Lotes (Batches)

Carregar as 5.552 matrizes simultaneamente causou estouro de memória (*Out of Memory*) no ambiente do Google Colab (~12 GB de RAM). Para solucionar este gargalo, implementamos um pipeline de dados otimizado com a API `tf.data` e o método `image_dataset_from_directory`:

- Processamento em Lotes (Batch Size = 32): O pipeline lê e processa apenas 32 imagens por vez sob demanda (*Lazy Loading*), descarregando a RAM logo após o envio dos dados para a GPU.
- Uso de `prefetch`: Aplicamos o método `.prefetch(tf.data.AUTOTUNE)`, que prepara o próximo lote de imagens em segundo plano enquanto a GPU treina o lote atual, eliminando gargalos de hardware.

4. Estratégia de Divisão e Organização dos Dados

Para mitigar problemas de viés cognitivo do modelo e garantir uma avaliação imparcial do classificador médico, a distribuição dos dados foi mapeada e dividida de forma estratificada nos seguintes subconjuntos:

1. **Conjunto de Treino (3.551 imagens):** Equivalente a 80% do diretório principal de desenvolvimento. Será utilizado para atualizar os pesos sinápticos e fazer a extração de *features* morfológicas pelo modelo.
2. **Conjunto de Validação (887 imagens):** Equivalente a 20% do diretório de desenvolvimento. Funciona como um termômetro durante a fase de treino para monitorar o *overfitting* (decoreba) e guiar o ajuste de hiperparâmetros sem viciar os pesos.
3. **Conjunto de Teste (1.114 imagens):** Isolado no diretório original `/test`. Este subconjunto foi mantido totalmente inédito e intocado pelo pipeline. Ele servirá estritamente para a geração da Matriz de Confusão, Acurácia, Precisão, Recall e F1-Score na Parte 2, garantindo uma validação real e segura para a aplicação hospitalar simulada.

5. Conclusão

Com o término da Parte 1, o pipeline de Engenharia de Dados do **Cardio-IA** encontra-se maduro, performático e em total conformidade com as boas práticas da ciência de dados. Ao resolver os problemas de estouro de memória utilizando arquiteturas de fluxos de lotes (`tf.data`), geramos tensores normalizados de formato estável e padronizado em (224, 224, 3). Os dados estão prontos e mastigados para a ingestão imediata e treinamento das redes neurais convolucionais customizadas e modelos de *Transfer Learning*.